

John Cheek
Alexander Card
CSC 481
10 November 2025

Milestone 4 Writeup

This milestone presented yet another major refactor of our game engine, but the event system was a welcome change of pace from a full refactor of our entity system. However, this one posed some more challenges for me than the last, particularly in the networking department.

Events in our engine are represented as strongly-typed structs that extend from a general event class. Each specific event type (input, collision, death, and spawn) also has a corresponding header file. This specific approach was chosen for clarity and extensibility. As per suggestions in the lecture slides, event registration uses a subscription model. Systems and components subscribe to the event types via the EventManager, which queues events and dispatches them in priority order. Figure 1 shows the header file for our EventManager, including the logic for handling event priority. For our general guideline, input events would have the highest priority, followed by collision events, then death and spawn events. To implement this new system, we redesigned our component system designed in milestone 3 to instead function using events. This did require those new systems to be updated again, but ultimately worked out as it allowed for easy code reuse and less manual implementation of events. For instance, the BoxCollider component now raises a collision event when a collision is detected. Now every game object using the box collider component will raise collision events and the event manager will dispatch them. This design enables easy addition of new event types and listeners making the engine extensible. For instance, adding a new gameplay mechanic would require defining a new event type and subscribing to the corresponding handler without modifying the core engine logic. Since we added networking in milestone 2 we have used this server centric approach

where the server uses the entity manager and contains all the most updated game states. Clients send input and event requests to the server where they are processed, added to the event manager, and dispatched via the network. This uses the request-reply pattern where the client sends a request in the form of an event command. Figure 2 shows the logic for the client creating an event command and figure 3 shows the server handling the input event command. Our main design philosophy for this milestone was modularity and extensibility. We aimed to make the systems as general as possible, by using components to communicate with engine systems. This allows for new engine features to be added by introducing new components and event types without rewriting existing code.

For my individual game project I decided to build upon my Milestone 2 game 'Falling Infinite' using the new events, while still creating a game that made sense in multiplayer and wasn't just another platformer level. Given the two large refactors since the original iteration of the game, I was unable to reuse any of my code from Milestone 2, but the ECS system made porting the desired logic over rather straightforward and, as a bonus, omitted the need to create brand new entity type for the coins and skulls. I also figured this game would be the best example to showcase the new event types. Input events for player movement, collision events for resolving interaction with the coins, and, to showcase death and spawn events, I wanted to introduce a new gameplay element. Now instead of coins falling from the sky, most of the objects that fall are 'skulls' which, if a player collides with them, will decrease their score and cause a death (and subsequent spawn) event while others are coins which increase the player's score. There is also another new mechanic where the game will spawn skulls/coins slowly and gradually ramp up to having them spawn more frequently, causing more hazards and opportunities to score. The input events are rather straight forward, they are created and sent to

the server whenever a player presses a movement key. Figure 4 shows the server console reporting input events from multiple players. The skulls and coins both use collision events to detect collisions and then handle their resolution, and the skulls take it a step further and trigger a death event when a collision event occurs. Figure 5 shows the client.cpp code where the client subscribes to the Collision events. This subscription code triggers whenever a collision event is raised from the EventManager, then handles the proper logic client side, and in the case of skulls, sends a death event to the server. For the spawn events, they are created once a death event is raised. They use our component system from the last milestone to reset the player back to the nearest spawn point when a spawn event occurs. Figure 6 shows the game in action with two connected players and debug textures enabled to display the spawn zones.

Overall, this was a slightly easier milestone than the previous one but the process of designing and implementing a new event system was very interesting and felt like a big step towards a more complete game engine. It was a difficult but rewarding challenge to reuse our component system from the last milestone and incorporate it into the new event system. The biggest challenge for me personally came from networking when making sure events were synchronized over the network and all connected clients. While the basic work to implement communication between client and server was straightforward, it presented many unusual bugs which were difficult to sort out. I was able to overcome them by taking a step back, looking at the network model as a whole, and putting faith in the new systems. If I could start over, I would honestly remove all the prior networking code that was specific to the old systems, keep the skeleton of the request-reply pattern, and rebuild networking from the ground up only relying on the new systems. While this may have been more difficult, I feel that ultimately it would have led to less unusual technical bugs, more consistent behavior, and a better optimized engine.

```

class EventManager {
public:
    using Listener = std::function<void(const Event&);>;
    using ListenerId = std::uint64_t;

    explicit EventManager(Timeline& timeline);

    // Registration
    ListenerId subscribe(EventType type, Listener cb);
    void unsubscribe(EventType type, ListenerId id);

    // Raising
    void raise(Event e); // queues event (timestamp auto-filled if <= 0)

    // Handling
    void dispatch(); // drains queue in priority order

private:
    struct QItem {
        Event event;
        std::uint64_t seq; // monotonically increasing tie-breaker
    };

    struct Compare {
        bool operator()(const QItem& a, const QItem& b) const {
            // lower priority enum value = higher priority
            if (a.event.priority != b.event.priority) {
                return static_cast<int>(a.event.priority) > static_cast<int>(b.event.priority);
            }
            // older timestamp first
            if (a.event.timestamp != b.event.timestamp) {
                return a.event.timestamp > b.event.timestamp;
            }
            // finally FIFO among identical priority/timestamp
            return a.seq > b.seq;
        }
    };

    Timeline& timeline_;
    std::priority_queue<QItem, std::vector<QItem>, Compare> queue_;
    std::unordered_map<EventType, std::unordered_map<ListenerId, Listener>> listeners_;
    ListenerId nextId_ = 1;
    std::uint64_t nextSeq_ = 1;
};

```

Figure 1: The header file for EventManager.cpp. Includes the comparator for event priority.

```

void send_event(zmq::socket_t& reqSock, const std::string& playerName, const std::string& eventType, const std::string& eventData) {
    std::ostringstream ss;
    ss << "EVENT|" << playerName << "|||"
        << eventType << "|" << eventData;
    std::string msg = ss.str();
    zmq::message_t request(msg.size());
    memcpy(request.data(), msg.data(), msg.size());
    try {
        std::lock_guard<std::mutex> lock(reqMutex);
        reqSock.send(request, zmq::send_flags::none);
        zmq::pollitem_t items[] = { { static_cast<void*>(reqSock), 0, ZMQ_POLLIN, 0 } };
        zmq::poll(items, 1, 100);
        if (items[0].revents & ZMQ_POLLIN) {
            zmq::message_t reply;
            reqSock.recv(reply, zmq::recv_flags::none);
        }
    }
    catch (const zmq::error_t& e) {
        std::cerr << "send_event zmq error: " << e.what() << std::endl;
        // Swallow to avoid terminating the client; caller can retry later
    }
}

```

Figure 2: Client side logic for sending an event command to the server

```

else if (cmd == "EVENT" && !eventType.empty()) {
    std::cout << "[Server] EVENT command received from " << playerName
    << " : eventType=\"" << eventType << "\" eventData=\"" << eventData << "\" << std::endl;

    if (eventType == "Input") {
        // Example: eventData = "MoveLeft:1"
        auto sep = eventData.find(':');
        if (sep != std::string::npos) {
            std::string action = eventData.substr(0, sep);
            bool pressed = eventData.substr(sep + 1) == "1";
            InputAction::Kind kind;
            if (action == "MoveLeft") kind = InputAction::Kind::MoveLeft;
            else if (action == "MoveRight") kind = InputAction::Kind::MoveRight;
            else if (action == "Jump") kind = InputAction::Kind::Jump;
            else kind = InputAction::Kind::None;

            std::lock_guard<std::mutex> lock(entityMutex);
            Entity* playerEntity = EntityManager::getInstance().findEntityByName(playerName);
            if (playerEntity) {
                float moveSpeed = 300.0f;
                if (kind == InputAction::Kind::MoveLeft && pressed) {
                    playerEntity->x -= moveSpeed * gameTimeline.getDeltaTime();
                }
                else if (kind == InputAction::Kind::MoveRight && pressed) {
                    playerEntity->x += moveSpeed * gameTimeline.getDeltaTime();
                }
                else if (kind == InputAction::Kind::Jump && pressed) {
                    if (playerEntity->type == "PLAYER") {
                        playerEntity->y += 150.0f;
                    }
                }

                eventManager.raise(Event{
                    EventType::Input,
                    EventPriority::High,
                    0.0f,
                    EventPayload{ InputAction{ kind, pressed } }
                });
                reply_str = "EVENT_OK";
            }
        }
    }
    else if (eventType == "Death") {
        // Parse eventData: "PlayerName,DeathZoneName"
        auto sep = eventData.find(',');
        std::string victimName = eventData.substr(0, sep);
        std::string deathZoneName = (sep != std::string::npos) ? eventData.substr(sep + 1) : "";

        std::lock_guard<std::mutex> lock(entityMutex);
        Entity* victim = EntityManager::getInstance().findEntityByName(victimName);
    }
}

```

Figure 3: Server side logic for processing Input event commands and (partially) Death event commands.

```

[Server] EVENT command received from P2: eventType="Input" eventData="MoveRight:1"
[Server] EVENT command received from P2: eventType="Input" eventData="MoveLeft:1"
[Server] EVENT command received from P2: eventType="Input" eventData="MoveRight:0"
[Server] EVENT command received from P2: eventType="Input" eventData="Jump:1"
[Server] EVENT command received from P2: eventType="Input" eventData="Jump:0"
[Server] EVENT command received from P2: eventType="Input" eventData="MoveLeft:0"
[Server] EVENT command received from P1: eventType="Input" eventData="MoveRight:1"
[Server] EVENT command received from P1: eventType="Input" eventData="Jump:1"
[Server] EVENT command received from P1: eventType="Input" eventData="Jump:0"
[Server] EVENT command received from P1: eventType="Input" eventData="Jump:1"
[Server] EVENT command received from P1: eventType="Input" eventData="Jump:0"
[Server] EVENT command received from P1: eventType="Input" eventData="Jump:1"
[Server] P2 hit a skull! score: 1
[Server] EVENT command received from P2: eventType="Death" eventData="P2,skull17"
[Server] Player P2 respawned at SpawnA
[Server] EVENT command received from P1: eventType="Input" eventData="MoveLeft:1"
[Server] EVENT command received from P1: eventType="Input" eventData="Jump:0"
[Server] EVENT command received from P1: eventType="Input" eventData="MoveRight:0"
[Server] EVENT command received from P1: eventType="Input" eventData="Jump:1"
[Server] EVENT command received from P1: eventType="Input" eventData="Jump:0"
[Server] EVENT command received from P1: eventType="Input" eventData="MoveLeft:0"
[Server] EVENT command received from P1: eventType="Input" eventData="MoveRight:1"
[Server] EVENT command received from P1: eventType="Input" eventData="MoveRight:0"
[Server] P1 collected a coin! score: 5
[Server] P2 hit a skull! score: 0
[Server] EVENT command received from P2: eventType="Death" eventData="P2,skull21"
[Server] Player P2 respawned at SpawnA

```

Figure 4: Server console log displaying input event commands and death event commands from multiple players.

```

eventManager.subscribe(EventType::Collision, [&](const Event& e) {
    auto* info = std::get_if<CollisionInfo>(&e.payload);
    if (!info) return;
    Entity* a = info->a;
    Entity* b = info->b;

    Entity* player = nullptr;
    Entity* coin = nullptr;
    Entity* skull = nullptr;

    if (a && b) {
        if (a->hasTag("PLAYER") && b->hasTag("COIN")) { player = a; coin = b; }
        else if (b->hasTag("PLAYER") && a->hasTag("COIN")) { player = b; coin = a; }
        else if (a->hasTag("PLAYER") && b->hasTag("SKULL")) { player = a; skull = b; }
        else if (b->hasTag("PLAYER") && a->hasTag("SKULL")) { player = b; skull = a; }
    }

    // Coin collection
    if (player && coin && player->name == playerName) {
        localScore += 3;
        std::cout << "[Client] Collected a coin! Local score: " << localScore << std::endl;
        // Remove coin locally
        auto& entities = EntityManager::getInstance().entities;
        auto it = std::find(entities.begin(), entities.end(), coin);
        if (it != entities.end()) {
            delete coin;
            entities.erase(it);
        }
    }

    // Skull collision triggers death event
    if (player && skull && player->name == playerName) {
        localScore -= 1;
        std::ostringstream deathData;
        deathData << player->name << ", " << skull->name;
        send_event(reqSock, playerName, "Death", deathData.str());
        localPlayerNeedsRespawn = true;
    }
});

```

Figure 5: Server side logic in Falling Infinite 2 for handling collision events between skulls/coins



Figure 6: Gameplay screenshot of Falling Infinite 2 with two players, skull and coin objects, and spawn zones. The player on the left has just received a death and subsequent spawn event. The player on the right is using input events to collect coins and avoid skulls